Lab - 1: **Experiment No: 1**

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the `hashlib` library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Program #1: Python program that uses the hashlib library to create the hash of a given string

```
import hashlib

def create_hash(string):
    hash_object = hashlib.sha256()
    hash_object.update(string.encode('utf-8'))
    hash_string = hash_object.hexdigest()
    return hash_string

input_string = input("Enter a string: ")
hash_result = create_hash(input_string)

print("Hash:", hash_result)
Enter a string: Mahek
Hash: acab4a512ebc7a16474db8768c165ffd91eec750e1d9f7eecbe870b70bb3ec39
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
Enter a string: Mahek
Enter the nonce: 54
```

```
Hash Code:
1d2904d2a986df0be9e3469c18784ee1024709afb3218f3aeca6800c908cfcff
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash starts with required zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = input("Enter the input string: ")
num_zeros = int(input("Enter the number of leading zeros: "))

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the result
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
Enter the input string: Mahek
Enter the number of leading zeros: 6
Hash: 0000002a94772b34e2e500630e47675c543c892b92493f1a0e99b2eca29b4283
```

```
Input String: Mahek
Leading Zeros: 6
Expected Nonce: 4238140
```

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def hash_data(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()


def build_merkle_tree(transactions):

    if len(transactions) == 0:
        return None

    # Step 1: Hash all transactions first
    transactions = [hash_data(transaction) for transaction in
transactions]

    print("\n--- Transaction Hashes ---")
    for i, transaction_hash in enumerate(transactions):
        print(f"T{i + 1}: {transaction_hash}")

    level = 1

    # Construct the Merkle Tree
    while len(transactions) > 1:

        print(f"\n--- Merkle Tree Level {level} ---")

        # If odd number of hashes, duplicate the last hash
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])
            print("Last hash duplicated because the number of hashes is
odd.")

        new_transactions = []

        for i in range(0, len(transactions), 2):
```

```
            # Combine two hashes
            combined = transactions[i] + transactions[i + 1]

            # Hash the combined hashes
            hash_combined = hash_data(combined)

            new_transactions.append(hash_combined)

            print(f"Hash {i // 2 + 1}:")
            print(f"  Hash 1: {transactions[i]}")
            print(f"  Hash 2: {transactions[i + 1]}")
            print(f"  Combined Hash: {hash_combined}")

        transactions = new_transactions
        level += 1

    return transactions[0]


# Sample Transactions
transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Roo -> Bob : $50"
]

# Construct Merkle Tree
merkle_root = build_merkle_tree(transactions)

# Print Merkle Root
print("\n================================")
print("Merkle Root Hash:", merkle_root)
print("================================")
--- Transaction Hashes ---
T1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
T2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
T3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
```

```
T4: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
T5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

--- Merkle Tree Level 1 ---
Last hash duplicated because the number of hashes is odd.
Hash 1:
  Hash 1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
  Hash 2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
  Combined Hash:
471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
Hash 2:
  Hash 1: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
  Hash 2: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
  Combined Hash:
a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Hash 3:
  Hash 1: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
  Hash 2: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
  Combined Hash:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

--- Merkle Tree Level 2 ---
Last hash duplicated because the number of hashes is odd.
Hash 1:
  Hash 1: 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
  Hash 2: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
  Combined Hash:
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
Hash 2:
  Hash 1: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
  Hash 2: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
  Combined Hash:
6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

--- Merkle Tree Level 3 ---
Hash 1:
  Hash 1: ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
  Hash 2: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
  Combined Hash:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

```
================================
Merkle Root Hash:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
================================
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.